

Esercizi 1 e 2 in C — Cassetta degli attrezzi completa

Tutte le system call, funzioni di libreria, header e idiomi che servono per risolvere **tutti** gli Esercizi 1 e 2 delle prove pratiche, con la mappatura prova-per-prova e i riferimenti alla teoria dei due PDF.

Vincolo della prova: per "eseguire un comando" si usa **sempre** `fork()` + `exec*()` + `wait()` a mano. Mai `system()`, mai `popen()`, mai `execlp("sh", "-c", ...)`. Compila sempre con `gcc -Wall -Wextra` e controlla **ogni** valore di ritorno con `errno` / `perror` (dispensa §2.9).

0. Header essenziali (cosa includere)

```
#include <stdio.h>      // printf, perror, fopen...
#include <stdlib.h>     // exit, malloc, free, realloc, atoi, getenv
#include <unistd.h>     // fork, exec*, read, write, close, lseek, dup2, getpid, getcwd, readlink
#include <sys/wait.h>   // wait, waitpid, WIFEXITED, WEXITSTATUS, WIFSIGNALED
#include <sys/stat.h>   // stat, lstat, fstat, struct stat, S_IF*, S_IS*, chmod, mkdir
#include <sys/types.h>  // pid_t, off_t, ssize_t, ino_t
#include <fcntl.h>      // open, O_RDONLY/O_WRONLY/O_CREAT/O_TRUNC/O_APPEND, openat
#include <dirent.h>     // opendir, readdir, closedir, scandir, fdopendir, struct dirent
#include <string.h>     // strcmp, strcpy, strcat, strdup, strlen, memcpy, strtok
#include <errno.h>      // errno, codici (ENOENT, EPERM...)
#include <signal.h>     // signal, sigaction, kill, sigqueue, sigemptyset
#include <limits.h>     // PATH_MAX
```

Header "speciali" che servono solo in alcune prove: - `#include <sys/inotify.h>` → inotify (2021.06.24, 2021.09.16, 2024.06.24) - `#include <sys/timerfd.h>` → timerfd (2023.06.14) - `#include <poll.h>` → poll (2024.09.11; teoria §6.3) - `#include <dlfcn.h>` → dlopen/dlsym (2021.07.15) — linkare con `-ldl` - `#include <time.h>` → clock_gettime, struct timespec (2022.09.07, 2023.06.14)

1. Processi: fork + exec + wait (lo schema base)

Teoria: dispensa §2.3–2.9 (fork, exit, wait, execve, errno) e §2.8.2 (mini-shell).

```
pid_t pid = fork();           // 0 nel figlio, >0 nel padre (PID figlio), -1 errore
if (pid < 0) { perror("fork"); exit(1); }
if (pid == 0) {               // FIGLIO
    // qui eventuali dup2() per ridirigere stdin/stdout
    execvp(file, argv);        // se ha successo NON ritorna mai
    perror("exec"); _exit(127); // raggiunta solo se exec fallisce
}
// PADRE
int status;
waitpid(pid, &status, 0);     // aspetta QUEL figlio
if (WIFEXITED(status)) printf("exit %d\n", WEXITSTATUS(status));
if (WIFSIGNALED(status)) printf("ucciso da segnale %d\n", WTERMSIG(status));
```

La famiglia `exec*` (dispensa §2.8.3):

Funzione	Quando usarla
<code>execve(path, argv, envp)</code>	syscall base, tutto esplicito (path assoluto + environment)
<code>execv(path, argv)</code>	path assoluto, eredita l'environment
<code>execvp(file, argv)</code>	cerca <code>file</code> nel <code>PATH</code> , eredita l'environment (la più comoda)
<code>execlp(file, argv, ..., NULL)</code>	argomenti come lista (es. <code>execlp("ls", "ls", NULL)</code>)

Regole d'oro: `argv` deve terminare con `NULL`, `argv[0]` è il nome del programma. `_exit()` nel figlio (non `exit()`) per non fare doppio flush dei buffer ereditati (dispensa §2.4).

- **Aspettare più figli** (pipe, parallelismo): un `wait(&status)` per ogni figlio.
- **WEXITSTATUS** estrae gli 8 bit bassi del codice di uscita (dispensa §2.6).

Esercizi: 2022.09.07 (`rilancia` : loop di fork+exec+wait, controlla `WIFEXITED` && `WEXITSTATUS==0` && `!WIFSIGNALED`), 2024.02.14/2024.06.24 (esegui i file trovati), 2021.06.24/2021.09.16 (esegui i file comparsi), 2023.05.24/2024.05.30 (clona processo).

2. Tempo di esecuzione (2022.09.07 es.2)

```
struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);
// ... fork + waitpid ...
clock_gettime(CLOCK_MONOTONIC, &t1);
double sec = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec)/1e9;
if (sec < 1.0) { /* non rilanciare */ }
```

3. File: open / read / write / close / lseek

Teoria: dispensa §4.1-4.3 (file descriptor, syscall fondamentali, lseek e sparse file).

```
int fd = open(path, O_RDONLY); // apre esistente
int fd = open(path, O_WRONLY|O_CREAT|O_TRUNC, 0666); // crea/azzerà
int fd = open(path, O_WRONLY|O_CREAT|O_APPEND, 0666); // append

ssize_t n;
char buf[4096];
while ((n = read(fd, buf, sizeof buf)) > 0) // n==0 -> EOF, n<0 -> errore
    write(fdout, buf, n);
close(fd);

off_t pos = lseek(fd, offset, SEEK_SET); // SEEK_SET/SEEK_CUR/SEEK_END
off_t size = lseek(fd, 0, SEEK_END); // dimensione del file
```

- **Copia di file** (modello §4.2.1): loop read→write con buffer.
- **lseek oltre la fine** crea un "buco" (sparse file, §4.3).
- **pread / pwrite** (offset esplicito, atomici): ideali per la copia parallela e l'inversione in-place.

```
ssize_t pread (int fd, void *buf, size_t n, off_t off);
ssize_t pwrite(int fd, const void *buf, size_t n, off_t off);
```

Esercizi: 2023.02.16 (**pcp** : ogni figlio **lseek / pread / pwrite** sulla propria fetta; dimensione totale con **lseek(...,SEEK_END)** o **fstat**), 2024.07.23 t2 (**inverti** in-place: due offset che convergono, **pread / pwrite**), copie varie (2022.01.18, 2023.07.20).

4. Ridirezione con dup2 e pipe

Teoria: dispensa §4.4 (dup/dup2/dup3), §4.6 (pipe), §4.6.2 (**ls | sort -r**).

```
// Ridirigere stdout su un file (come '> file')
int fd = open(path, O_WRONLY|O_CREAT|O_TRUNC, 0666);
dup2(fd, STDOUT_FILENO); // ora fd 1 punta al file
close(fd);

// Pipe tra due processi (cmd1 | cmd2)
int fd[2];
pipe(fd); // fd[0]=lettura, fd[1]=scrittura
// figlio 1: dup2(fd[1], STDOUT_FILENO); chiude entrambi; exec cmd1
// figlio 2: dup2(fd[0], STDIN_FILENO); chiude entrambi; exec cmd2
// PADRE: close(fd[0]); close(fd[1]); due wait() <-- FONDAMENTALE
```

Punto critico (dispensa §4.6.2, nota): il padre **deve chiudere entrambi i capi**, altrimenti il lettore non riceve mai EOF e resta bloccato.

- **Catena di N comandi** (2023.01.19 es.2 / 2024.07.23 stile pipe): array di pipe in un loop; ogni stadio prende **fd[i-1][0]** come stdin e **fd[i][1]** come stdout.

Esercizi: 2023.01.19 (**stdin2pipe**), 2021.06.24/2021.09.16 es.2 (output dell'eseguibile ridiretto su un file via **dup2**).

5. Metadati: stat / lstat / fstat e struct stat

Teoria: dispensa §3.2 (**struct stat** , **st_mode & 07777**), §3.3 (**stat** vs **lstat**).

```

struct stat st;
stat (path, &st);    // SEGUE i symlink
lstat(path, &st);    // NON segue i symlink (usalo per VEDERE il link)
fstat(fd, &st);      // su un fd già aperto

st.st_ino    // numero di inode    -> identità file / hard link
st.st_dev    // device (file unico = coppia (st_dev, st_ino))
st.st_size   // dimensione in byte -> 2025.01.21, 2025.02.13
st.st_nlink  // numero di hard link
st.st_mode   // tipo + permessi
st.st_mtime  // ultima modifica (time_t) -> 2023.07.20 es.2, 2026.01.12
st.st_uid    // proprietario      -> /proc, processi utente

// Tipo del file (dispensa §3.2)
S_ISREG(st.st_mode) S_ISDIR(st.st_mode) S_ISLNK(st.st_mode)
S_ISFIFO(st.st_mode) S_ISCHR(st.st_mode) S_ISBLK(st.st_mode)

// Permessi
st.st_mode & 07777          // bit di permesso in ottale

```

- **Hard link dello stesso file:** `a.st_ino == b.st_ino && a.st_dev == b.st_dev`.
- **Stessa dimensione ma non hard link** (2025.02.13): `st_size` uguale ma inode diverso.
- **Eseguibile?** `access(path, X_OK)` (più diretto del controllo sui bit).

6. Directory: opendir / readdir / scandir / openat+fdopendir

Teoria: dispensa §3.4 (dirent, readdir, scandir).

```

DIR *d = opendir(path);
struct dirent *de;
while ((de = readdir(d)) != NULL) {
    if (!strcmp(de->d_name, ".") || !strcmp(de->d_name, "..")) continue;
    // de->d_name = nome (non il path), de->d_ino, de->d_type (DT_DIR, DT_REG, DT_LNK)
}
closedir(d);

```

Visita ricorsiva manuale (lo scheletro per ~metà delle prove)

```

void walk(const char *path) {
    DIR *d = opendir(path);
    if (!d) { perror(path); return; }
    struct dirent *de;
    while ((de = readdir(d))) {
        if (!strcmp(de->d_name, ".") || !strcmp(de->d_name, "..")) continue;
        char child[PATH_MAX];
        snprintf(child, sizeof child, "%s/%s", path, de->d_name);
        struct stat st;
        lstat(child, &st);          // lstat: non segue i link
        if (S_ISDIR(st.st_mode)) walk(child);    // ricorsione
        else { /* elabora il file */ }
    }
    closedir(d);
}

```

scandir (lettura ordinata, con filtro)

```

struct dirent **namelist;
int n = scandir(path, &namelist, filtro, alphasort);    // alphasort = ordine alfabetico
for (int i = 0; i < n; i++) { ...; free(namelist[i]); }
free(namelist);

```

openat + fdopendir (richiesto da 2022.07.22 es.2: niente concatenazione di stringhe)

```

void walkfd(int dirfd) {
    DIR *d = fdopendir(dirfd);          // trasforma un fd in DIR*
    struct dirent *de;
    while ((de = readdir(d))) {
        if (!strcmp(de->d_name, ".") || !strcmp(de->d_name, "..")) continue;
        struct stat st;
        fstatat(dirfd, de->d_name, &st, AT_SYMLINK_NOFOLLOW);
        if (S_ISDIR(st.st_mode)) {
            int sub = openat(dirfd, de->d_name, O_RDONLY|O_DIRECTORY);
            walkfd(sub);                // scende senza costruire path
        }
    }
    closedir(d);    // chiude anche dirfd
}

```

Esercizi: tutte le prove su alberi/link (vedi mappa §15); 2022.07.22 (`tree`), 2023.09.14 (`vreaddir` : leggi i nomi e mettili in un vettore — vedi §11).

7. Link simbolici e fisici

```
ssize_t k = readlink(link, buf, sizeof buf - 1); // legge il TARGET (NON null-terminato!)
buf[k] = '\0';
char *abs = realpath(path, NULL); // path assoluto canonico (risolve i symlink); free()
symlink(target, linkpath); // crea un symlink
link(oldpath, newpath); // crea un HARD link
unlink(path); // rimuove un nome (file o link)
rename(old, new); // sposta/rinomina ATOMICO
```

Idiomi chiave (riferiti agli esercizi): - **Symlink vs hard link di f** (2020.01.17 `searchlink`): per ogni voce fai `lstat`. Se `S_ISLNK` e `realpath(voce)==realpath(f)` → symlink. Se `S_ISREG` e stesso inode di f → hard link. - **Symlink → assoluto** (2020.02.21, 2022.01.18): `t=readlink`, se non assoluto `realpath` per ottenerlo, `unlink` + `symlink(assoluto, link)`. - **Symlink nidificato** (2022.02.17): leggi il target con `readlink`, poi `lstat` del target; se anch'esso `S_ISLNK` è nidificato → `unlink` (es.1) o sostituisci con hard link al file finale (es.2). - **Invertire un symlink** (2022.06.08 `invsymlink`): con `realpath` ottieni i due path, poi sposta il file reale al posto del link (`rename`) e crea un `symlink` nella vecchia posizione. La traccia stessa suggerisce `realpath(3)`. - **Symlink relativo vs assoluto** (2023.06.14): `readlink` e controlla se inizia con `/`.

Teoria: §3.3-3.4; appunti esercitazioni (tabella `link` / `symlink` / `readlink`, inode).

8. Segnali

Teoria: dispensa §5 (kill, signal, sigaction, SA_SIGINFO, siginfo_t).

```
// Inviare un segnale (2023.01.23 es.2)
kill(pid, SIGUSR1);

// Handler classico
void handler(int signo) { ... }
signal(SIGUSR1, handler);

// Handler avanzato con dati (SA_SIGINFO) — riceve mittente e valore
void handler(int signo, siginfo_t *si, void *ctx) {
    si->si_pid; // PID del mittente
    si->si_value.sival_int; // valore intero allegato
    si->si_value.sival_ptr; // oppure puntatore (8 byte su 64 bit)
}
struct sigaction sa = {0};
sa.sa_sigaction = handler;
sa.sa_flags = SA_SIGINFO;
sigemptyset(&sa.sa_mask);
sigaction(SIGUSR1, &sa, NULL);

// Inviare un segnale CON un valore (lato mittente)
union sigval v; v.sival_int = qualcosa;
sigqueue(pid_dest, SIGUSR1, v);

// Attendere efficientemente i segnali
for (;;) pause(); // o sigsuspend(&mask) per evitare race
```

Esercizi: 2022.06.22 / 2026.02.09 (`tx` / `rx`, `sigtx` / `sigrx` : impacchetta ≤8 byte nel `sival` e li invia con `sigqueue`; lato rx `SA_SIGINFO` legge `si_value`), 2025.06.24 (`semSEND` / `semrcv` : trasferimento **bit a bit** con `SIGUSR1` / `SIGUSR2`, ricomponendo i byte; distingui i mittenti con `si_pid`), 2023.01.23 es.2 (`fifosig` : leggi `pid segnale` e fai `kill`).

9. inotify (NON nei PDF — serve il man)

```
int fd = inotify_init1(0);
int wd = inotify_add_watch(fd, dir, IN_CLOSE_WRITE | IN_MOVED_TO);
char buf[4096] __attribute__((aligned(8)));
for (;;) {
    ssize_t len = read(fd, buf, sizeof buf); // BLOCCANTE: si sveglia agli eventi
    for (char *p = buf; p < buf + len; ) {
        struct inotify_event *e = (struct inotify_event *)p;
        if (e->len) {
            // e->name = nome del file comparso in dir
        }
        p += sizeof(struct inotify_event) + e->len;
    }
}
```

- Eventi utili: `IN_CLOSE_WRITE` (file finito di scrivere), `IN_MOVED_TO`, `IN_CREATE`.
- Dopo aver elaborato il file (copiarlo / eseguirlo) → `unlink`.

Esercizi: 2021.06.24 (**dircat** : appendi nome+contenuto a F, poi cancella; es.2: se eseguibile, appendi l'**output** con fork+exec+dup2), 2021.09.16 (**execname** : il **nome** del file è il comando → split in argv ed esegui), 2024.06.24 (**inotirun** : il **contenuto** del file è exe + una riga per argv → esegui).

10. timerfd e poll (timerfd NON nei PDF; poll sì, §6.3)

```
int tfd = timerfd_create(CLOCK_MONOTONIC, 0);
struct itimerspec its = {0};
its.it_value.tv_sec = 1; its.it_value.tv_nsec = 100000000; // prima scadenza
its.it_interval = its.it_value; // periodica
timerfd_settime(tfd, 0, &its, NULL);
uint64_t exp;
read(tfd, &exp, sizeof exp); // si sblocca a ogni scadenza; exp = n. scadenze

// poll su più fd (timer multipli / timeout)
struct pollfd pfd[N];
pfd[i].fd = tfd_i; pfd[i].events = POLLIN;
poll(pfd, N, -1); // -1 = attesa infinita (dispensa §6.3)
if (pfd[i].revents & POLLIN) { ... }
```

Esercizi: 2023.06.14 (**tfdtest** : una stringa a intervalli; es.2 più timer con **poll**), 2024.09.11 (**timeout** : fork+exec del comando, **pidfd_open** sul figlio + **timerfd** al timeout, **poll** su entrambi: scade il timer → **kill** ; pronto il pidfd → figlio finito).

pidfd_open (NON nei PDF): `int pfd = syscall(SYS_pidfd_open, pid, 0);` poi monitorabile con **poll** (diventa **POLLIN** alla morte del processo).

11. Memoria dinamica e stringhe (2023.09.14 vreaddir)

```
char **v = malloc(cap * sizeof(char *));
v[i] = strdup(de->d_name); // copia allocata della stringa
v = realloc(v, newcap * sizeof(char *));
v[n] = NULL; // terminatore stile argv/envp
// liberare: for(i) free(v[i]); free(v);
```

- **es.2 (un solo malloc / un solo free)**: calcola in due passate la dimensione totale (numero di puntatori + somma dei **strlen+1**), alloca **un unico blocco**, metti i puntatori in testa e copia le stringhe nella coda dello stesso blocco. Così basta un **free** .

Funzioni utili: **strdup** , **strcpy** / **strncpy** , **strcat** , **strlen** , **memcpy** , **strtok** (split del nome in argv per 2021.09.16), **snprintf** (comporre path senza overflow).

12. /proc — informazioni sui processi (2023.05.24, 2024.05.30)

```
// /proc/<pid>/cmdline : argv separati da '\0'
int fd = open("/proc/1234/cmdline", O_RDONLY);
ssize_t n = read(fd, buf, sizeof buf); // i campi sono terminati da '\0'

// /proc/<pid>/exe : symlink all'eseguibile reale
readlink("/proc/1234/exe", path, sizeof path);

// /proc/<pid>/environ : environment (variabili separate da '\0')
// /proc/<pid>/cwd : symlink alla directory corrente del processo
// proprietario del processo: stat("/proc/1234", &st) -> st.st_uid
```

Per scorrere tutti i processi: `opendir("/proc")` e tieni solo i nomi tutti-cifre.

Esercizi: 2023.05.24 (**pidcmd** : confronta **cmdline** con argv → stampa i PID; es.2 clona leggendo **exe** / **cmdline** / **environ** / **cwd** e **execve**), 2024.05.30 (**cloneproc** : **fork** + **execve** con exe+argv da **/proc** ; es.2 aggiunge **chdir** su **cwd** e l' **environ**).

13. dlopen — librerie dinamiche (2021.07.15; NON nei PDF, linkare -ldl)

```
#include <dlfcn.h>
void *h = dlopen(realpath_della_lib, RTLD_NOW); // serve il PATH ASSOLUTO
int (*f)(int, char **) = dlsym(h, "main"); // indirizzo della funzione
int rc = f(argc, argv);
dlclose(h);
```

- es.2 (**autolancia**): distingui libreria dinamica vs eseguibile ELF leggendo l'header, poi **dlopen** oppure **fork** + **exec** .

14. getopt — parsing opzioni (2023.02.16, 2025.02.13)

```
#include <unistd.h>
int opt, jobs = 2;
while ((opt = getopt(argc, argv, "j:sl")) != -1) {
    switch (opt) {
        case 'j': jobs = atoi(optarg); break;    // 'j:' = opzione con argomento
        case 's': /* ... */ break;
        case 'l': /* ... */ break;
        default: /* uso */ ;
    }
}
// gli argomenti non-opzione iniziano da argv[optind]
```

Esercizi: 2023.02.16 (`pcp -j 8`), 2025.02.13 (`ckfile` con `-s / -l / -p N`), 2020.01.17 es.2 (`-c / -l / -s`).

15. Creare directory e file (catalogazione, copia di alberi)

```
mkdir(path, 0777);           // crea una directory
chdir(path);                 // cambia directory corrente
getcwd(buf, sizeof buf);     // directory corrente (per path assoluti, 2025.05.29)
chmod(path, 0644);           // cambia permessi
```

Esercizi: 2023.07.20 (`cp -rl` : ricrea l'albero con `mkdir` + `link` ; es.2 confronta `st_mtime` per scegliere link vs copia), 2024.07.23 t1 (dir `...` + `rename` + `symlink` relativo), 2024.02.14 es.2 (`chdir` nella dir del file prima di eseguirlo).

16. Mappa rapida prova → strumenti C

Prova	Es.1 / Es.2	Strumenti chiave
2020.01.17	searchlink / opzioni -c -l -s	walk+ <code>lstat</code> , <code>st_ino</code> , <code>readlink</code> / <code>realpath</code> , <code>getopt</code> , <code>unlink</code> / <code>link</code> / <code>symlink</code>
2020.02.21	abslink / absls	<code>readlink</code> , <code>realpath</code> , <code>unlink</code> + <code>symlink</code>
2021.06.24	dircat (inotify) / output eseguibile	<code>inotify</code> , <code>open</code> / <code>read</code> / <code>write</code> O_APPEND, fork+exec+ <code>dup2</code> , <code>unlink</code>
2021.07.15	lancia (dlopen) / autolancia	<code>dlopen</code> / <code>dlsym</code> , <code>realpath</code> , header ELF
2021.09.16	execname (inotify) / output su file	<code>inotify</code> , <code>strtok</code> /split, fork+ <code>execvp</code> , <code>dup2</code>
2022.01.18	abssymlink / cpsymlink	walk, <code>readlink</code> / <code>realpath</code> , <code>symlink</code> , copia read/write
2022.02.17	nonest_symlink / nest2hard	<code>readlink</code> + <code>lstat</code> del target, <code>unlink</code> / <code>link</code>
2022.06.08	invsymlink / su intera dir	<code>realpath</code> , <code>rename</code> , <code>symlink</code> , walk
2022.06.22	tx/rx (sigqueue) / stringhe lunghe	<code>sigqueue</code> , <code>sigaction</code> +SA_SIGINFO, <code>si_value</code>
2022.07.22	tree / con openat+fdopendir	<code>opendir</code> / <code>readdir</code> ; es.2 <code>openat</code> + <code>fdopendir</code> + <code>fstatat</code>
2022.09.07	rilancia / non rilanciare se <1s	fork+ <code>execvp</code> + <code>waitpid</code> , <code>WIFEXITED</code> / <code>WEXITSTATUS</code> , <code>clock_gettime</code>
2023.01.19	stdin2pipe / N comandi	<code>pipe</code> , <code>dup2</code> , fork×N, array di pipe, wait×N
2023.01.23	fifotext / fifosig	<code>mkfifo</code> , <code>open</code> loop riapertura, EOF; es.2 <code>kill</code>
2023.02.16	pcp / -j N	<code>fstat</code> / <code>lseek</code> size, <code>pread</code> / <code>pwrite</code> , fork×N, <code>getopt</code>
2023.05.24	pidcmd / clona da /proc	/proc/*/cmdline , <code>execve</code> ; es.2 <code>exe</code> / <code>environ</code> / <code>cwd</code>
2023.06.14	tfdtest / multipli (poll)	<code>timerfd_create</code> / <code>settime</code> , <code>read</code> ; es.2 <code>poll</code>
2023.07.20	cpri / cprlt	walk, <code>mkdir</code> , <code>link</code> , copia; es.2 <code>st_mtime</code>
2023.09.14	vreaddir / un solo malloc	<code>opendir</code> / <code>readdir</code> , <code>strdup</code> / <code>malloc</code> ; es.2 blocco unico
2024.01.19	argsend/argrecv / pargrcv	concat con <code>\0</code> , ricostruzione argv, fork+ <code>execvp</code> ; es.2 <code>mkfifo</code>
2024.02.14	search_name / run_name	walk, <code>access(X_OK)</code> , magic ELF/ <code>#!</code> ; es.2 <code>chdir</code> + <code>execv</code>
2024.05.30	cloneproc / cloneproc+	/proc/pid/exe + <code>cmdline</code> , <code>execve</code> ; es.2 <code>chdir</code> + <code>environ</code>
2024.06.24	inotirun / inotimrun	<code>inotify</code> , parse righe→argv, fork+ <code>execv</code> , <code>unlink</code>
2024.07.23 t1	dir ... + symlink (rename atomico)	<code>mkdir</code> , <code>rename</code> , <code>symlink</code> relativo, <code>realpath</code> /relpath
2024.07.23 t2	inverti in-place / pinverti	<code>pread</code> / <code>pwrite</code> ai due estremi, fork×N
2024.09.11	timeout (poll+pidfd+timerfd) / riattiva	<code>pidfd_open</code> , <code>timerfd</code> , <code>poll</code> , <code>kill</code>
2025.01.21	samecont (dimensione) / symlink a f	walk, <code>st_size</code> , <code>st_ino</code> ; <code>readlink</code> / <code>realpath</code>
2025.02.13	ckfile -s/-l / -p N contenuto	<code>getopt</code> , <code>lstat</code> / <code>st_ino</code> , confronto byte
2025.05.29	search_prec / search_drec	<code>getcwd</code> +nome, confronto contenuto col path
2025.06.24	semsend/semrecv (bit a bit) / multi-mittente	<code>SIGUSR1</code> / <code>SIGUSR2</code> , ricomposizione byte, <code>si_pid</code>
2025.07.23	sha1dir / sha1diff	walk, <code>mkdir</code> , sha1 (libreria), confronto
2025.09.04	shalindex / shalupdate	<code>symlink</code> con nome=sha1; es.2 <code>lstat</code> mtime
2026.01.12	modifcmp / modif=	walk, <code>st_mtim</code> , <code>st_ino</code> , confronto tempi
2026.02.09	sigtx/sigrx / con ack	<code>sigqueue</code> , SA_SIGINFO, segnale di conferma

17. Insidie ricorrenti (dove si perdono punti)

- `stat` vs `lstat` : sui link usa SEMPRE `lstat` / `fstatat(...,AT_SYMLINK_NOFOLLOW)` , altrimenti vedi il bersaglio invece del link (dispensa §3.3).
- `readlink` non null-termina: scrivi a mano `buf[k]='\0'` usando il valore di ritorno.
- Padre che non chiude la pipe:** il lettore non riceve EOF e si blocca (§4.6.2).
- `exec` che ritorna: significa errore → sempre `perrot` + `_exit(127)` dopo l'exec.
- `_exit` vs `exit` nel figlio: usa `_exit` per non duplicare i buffer (§2.4).
- `argv` senza NULL finale: `exec*` fallisce o legge memoria a caso.
- Buffer per i path:** usa `PATH_MAX` e `snprintf` (mai `strcat` senza limiti).
- Controllo errori:** ogni syscall ritorna `-1` / `NULL` in caso di errore → controlla `errno` / `perrot` (§2.9). Le prove tolgono punti se manca.
- `realpath(path, NULL)` alloca con malloc → ricordati `free` .
- inotify/timerfd/pidfd/dlopen/sigqueue NON sono nei due PDF:** tienili a mente e apri il `man` durante la prova (consentito).

```
// gcc -Wall -Wextra -o prog prog.c (aggiungi -ldl per dlopen)
```